WORKSHEET — UNIT IV

Quantum Programming and Simulation Platforms

Course: Fundamentals of Quantum Computing

Unit: IV (5 Hrs / 300 min)

Topics: From Theory to Running Code; The Quantum SDK Landscape; Hands-On Qiskit Fundamentals; Simulators vs Real Hardware; Noise and Decoherence; Noise Mitigation and Circuit Compilation; Quantum Error Correction Basics; Integrated Lab Walkthrough

🎯 Section A — Learning Objectives

1. Describe a realistic NISQ-era workflow and compare the main quantum SDKs (Qiskit, Cirq, Q#, PennyLane, Braket) and the role of OpenQASM.

2. Build, draw, simulate and measure a quantum circuit in Qiskit, and read counts, histograms and state-vector output.

3. Explain why classical simulation scales as 2^n, how cloud QPUs are accessed, how to read backend properties, and why transpilation is needed.

4. Explain decoherence (T1, T2), gate, readout and crosstalk errors, the common noise channels, and the basics of density matrices and noise models.

5. Apply noise-mitigation techniques (measurement calibration, zero-noise extrapolation, probabilistic error cancellation, dynamical decoupling) and circuit-compilation strategies.

6. Explain quantum error correction: the 3-qubit bit-flip and phase-flip codes, the Shor code, stabilizers, the surface code, and the logical error rate of a repetition code.

🔑 Section B — Key Concepts

No.	Concept	Short Explanation
1	**NISQ-Era Workflow**	Write a circuit, transpile it, simulate it, run it on hardware, mitigate errors and interpret the results; devices are noisy and have limited qubits.
2	**Quantum SDKs**	Qiskit (IBM), Cirq (Google), Q# with the Azure Quantum Development Kit (Microsoft), PennyLane and Amazon Braket let programmers build and run circuits.
3	**OpenQASM**	A text format that serves as a common intermediate representation, so circuits can move between tools and devices.
4	**Qiskit Circuits and Aer**	A QuantumCircuit holds quantum and classical registers, gates and measurements; Aer simulators return state vectors, unitaries or counts.
5	**Simulators vs Real Hardware**	A simulator needs memory that grows as 2^n but is noiseless by default; cloud QPUs are real but noisy and shared through queues.
6	**Transpilation**	Rewrites a circuit into the backend's native gates and coupling map, inserting SWAPs and optimizing depth.
7	**Noise and Decoherence**	Interaction with the environment destroys quantum information: T1 is relaxation, T2 is dephasing, and gates and readout add errors.
8	**Noise Channels and Density Matrices**	Bit-flip, phase-flip, depolarizing and amplitude-damping channels describe errors; density matrices describe the resulting mixed states.
9	**Noise Mitigation**	Calibration matrices, zero-noise extrapolation, probabilistic error cancellation and dynamical decoupling reduce the effect of noise without full error correction.
10	**Quantum Error Correction**	Redundancy across many physical qubits and syndrome measurements detect and fix errors without measuring the data.

11	**Surface Code and Overhead**	A 2D lattice of data and ancilla qubits with a threshold theorem; many physical qubits are needed for each logical qubit.

📘 Section C — Important Terminology

Keyword	Meaning
NISQ	Noisy Intermediate-Scale Quantum: devices with tens to hundreds of noisy qubits and no full error correction.
SDK	Software development kit: the libraries and tools used to write and run quantum programs.
OpenQASM	An open text format for quantum circuits that serves as a common intermediate representation.
QuantumCircuit	The Qiskit object that holds registers, gates and measurements of a circuit.
Aer	Qiskit's high-performance simulator package, offering state-vector, unitary and sampling (QASM-style) simulation.
Shots	The number of times a circuit is run to collect measurement statistics (counts).
Backend	A simulator or real quantum device on which a circuit is run.
Coupling Map	A graph showing which pairs of physical qubits can interact directly with a two-qubit gate.
Native Gates	The basic gates a device implements directly; other gates are decomposed into them.
Transpilation	Rewriting a circuit to use the native gates and coupling map of a backend.
Decoherence	Loss of quantum behaviour through interaction with the environment.
T1 (Relaxation)	The time in which an excited $\|1\rangle$ state decays towards $\|0\rangle$ by energy loss.
T2 (Dephasing)	The time in which the phase relationship between $\|0\rangle$ and $\|1\rangle$ is lost; $T2 \le 2 \cdot T1$.
Readout Error	A measurement reports the wrong value, e.g. reading 0 for a qubit that was 1.
Crosstalk	Unwanted influence of operations on a qubit by its neighbouring qubits or control lines.
Bit-Flip / Phase-Flip	Channels that apply X (swap 0 and 1) or Z (flip the relative phase) with some probability.
Depolarizing Noise	A channel that replaces the state, with some probability, by a random (maximally mixed) state.
Amplitude Damping	A channel modelling energy loss, in which $\|1\rangle$ decays to $\|0\rangle$ (the T1 process).
Density Matrix	A matrix ρ that describes pure and mixed states; $\mathrm{Tr}(\rho) = 1$ and $\mathrm{Tr}(\rho^2) < 1$ for a mixed state.
Calibration Matrix	A matrix of measured readout probabilities used to correct measurement errors.
Zero-Noise Extrapolation	Running a circuit at several amplified noise levels and extrapolating the result back to zero noise.
Dynamical Decoupling	Applying pulse sequences to idle qubits to cancel the effect of dephasing.
Syndrome	The result of parity-check measurements that reveals where an error occurred without revealing the data.
Logical Qubit	A qubit encoded across many physical qubits so that errors can be detected and corrected.
Surface Code	A 2D error-correcting code with data and ancilla qubits that needs only nearest-neighbour interactions.

✅ Section D — Multiple-Choice Questions [Easy–Medium]

1. Which sequence describes a typical NISQ-era workflow?

A. Run on hardware → write → transpile → interpret
B. Mitigate → write → run → simulate
C. Write → transpile → simulate → run on hardware → mitigate → interpret
D. Interpret → write → transpile → run

2. What is OpenQASM used for?

A. A common intermediate representation for quantum circuits
B. A quantum computing chip
C. A noise model for simulators
D. An error-correcting code

3. How does the memory needed for a full state-vector simulation of n qubits grow?

A. Like n
B. Like n^2
C. Like 2n
D. Like 2^n

4. Which simulator mode returns the full list of amplitudes of the final state?

A. Sampling simulator that returns counts only
B. Statevector simulator
C. Readout calibration
D. Hardware queue

5. What does T1 describe?

A. Relaxation of $|1\rangle$ towards $|0\rangle$ through energy loss
B. Dephasing of the superposition
C. The error of the measurement
D. The speed of a gate

6. Which noise channel applies X to the qubit with probability p?

A. Phase-flip channel
B. Amplitude damping
C. Bit-flip channel
D. Depolarizing channel

7. A qubit in the maximally mixed state I/2 has purity $Tr(\rho^2)$ equal to …

A. 1
B. 1/2
C. 0
D. 2

8. Which technique runs a circuit at several amplified noise levels and extrapolates back to zero noise?

A. Dynamical decoupling
B. Measurement calibration
C. Transpilation
D. Zero-noise extrapolation

9. How does the 3-qubit bit-flip code encode $\alpha|0\rangle + \beta|1\rangle$?

A. As $\alpha|000\rangle + \beta|111\rangle$
B. As $\alpha|000\rangle + \beta|000\rangle$
C. As $(|000\rangle + |111\rangle)/\sqrt{2}$ for every state
D. As $\alpha|{+}{+}{+}\rangle + \beta|{-}{-}{-}\rangle$

10. If each qubit flips independently with probability p, what is the logical error probability of the 3-qubit repetition code with majority vote?

A. p^3
B. $3p^2 - 2p^3$
C. $3p$
D. p^2

✏️ Section E — Fill in the Blanks [Easy]

1. A common intermediate representation for quantum circuits is called Open_____.

2. Rewriting a circuit to use a device's native gates and coupling map is called _____.

3. The loss of phase coherence between $|0\rangle$ and $|1\rangle$ is characterized by the _____ time.

4. Measurement error mitigation uses a _____ matrix, built by preparing each basis state and recording what is measured.

5. In the 3-qubit bit-flip code, the parity-check results that locate the error are called the _____.

☑️ Section F — True or False [Easy]

1. Classically simulating n qubits needs memory that grows exponentially with n. (True / False)

2. The dephasing time T2 can be larger than $2 \cdot T1$. (True / False)

3. Quantum error correction may copy a qubit's state three times to create redundancy. (True / False)

4. Correcting X and Z errors is enough to correct any single-qubit error. (True / False)

5. A noiseless simulator reproduces the results of a real device exactly. (True / False)

🔗 Section G — Match the Following [Medium]

Column A	Column B (shuffled)
1. Qiskit	a. Microsoft's quantum language used with the Azure Quantum Development Kit
2. Cirq	b. Parity-check results that locate an error without revealing the data
3. Q#	c. IBM's open-source Python SDK with Aer simulators and cloud QPU access
4. PennyLane	d. Google's Python framework for circuits on NISQ hardware
5. Zero-noise extrapolation	e. 2D lattice code with data and ancilla qubits and a threshold near 1%
6. Syndrome	f. Hybrid quantum–classical programming with automatic differentiation
7. Surface code	g. Extrapolating results measured at amplified noise back to zero noise

📝 Section H — Short-Answer Questions [Medium]

1. Why does classical simulation of n qubits scale as 2^n? Estimate the memory for 30 qubits at 16 bytes per amplitude.

2. What is transpilation, and why is it needed before a circuit runs on a real device?

3. Define T1 and T2, and name three sources of error on a device other than decoherence.

4. Explain how measurement error mitigation works using a calibration matrix.

5. Explain how the 3-qubit bit-flip code encodes, detects and corrects a single bit-flip error.

📄 Section I — Descriptive Questions [Medium–Advanced]

1. Compare the quantum SDKs (Qiskit, Cirq, Q#, PennyLane, Braket), explain OpenQASM and the criteria for choosing an SDK, and describe how to build, draw, simulate and measure a 2-qubit circuit in Qiskit.

2. Compare simulators and real hardware: the 2^n scaling and its laptop and HPC limits, access to cloud QPUs (queues, jobs, backends), backend properties (qubit count, coupling map, gate errors, T1/T2), transpilation, and how to compare results.

3. Explain noise and its mitigation: T1 and T2, gate, readout and crosstalk errors, the common noise channels, density matrices, building a noise model, and the techniques of calibration matrices, zero-noise extrapolation, probabilistic error cancellation, dynamical decoupling and circuit compilation.

4. Explain quantum error correction: why classical methods do not transfer, the digitization of errors, the bit-flip and phase-flip codes, the Shor code, the stabilizer formalism, the surface code and threshold theorem, the physical-to-logical overhead, and the logical error rate of a repetition code.

💻 Section J — Practical / Application-Based Questions [Advanced]

Practical 1

Problem: Estimate the memory needed for state-vector simulation.

Task:

1. Define memory_gib(n) = $2^n \times 16$ bytes, converted to GiB (1 GiB = 2^{30} bytes), rounded to 3 decimal places.

2. Print memory_gib(n) for n = 20, 30 and 35.

3. Find and print the largest n whose state vector fits in 64 GiB of RAM.

Expected Output: *Four lines: 0.016, 16.0, 512.0, then 32.*

Practical 2

Problem: Correct readout errors with a calibration matrix.

Task:

1. A device reads 0 as 1 with probability 0.05 and reads 1 as 0 with probability 0.10, so the calibration matrix is A = [[0.95, 0.10], [0.05, 0.90]].

2. The measured probabilities are (0.61, 0.39). Compute the inverse of A (use the 2×2 determinant formula) and apply it to the measured vector.

3. Print the corrected probabilities rounded to 3 decimal places.

Expected Output: *One line: (0.6, 0.4).*

Practical 3

Problem: Compute the logical error rate of the 3-qubit repetition code.

Task:

1. Define logical_error(p) = $3p^2(1 - p) + p^3$, the probability that two or three of the three qubits flip.

2. For p = 0.01, 0.1 and 0.5, print a tuple (p, logical_error(p) rounded to 6 decimals, whether logical_error(p) < p).

3. State for which values of p the code helps.

Expected Output: *Three lines: (0.01, 0.000298, True), (0.1, 0.028, True), (0.5, 0.5, False); the code helps when $p < 0.5$.*

🗺️ Section K — Concept-Mapping Activity

QUANTUM PROGRAMMING AND SIMULATION

Complete the map below: fill in the missing sub-topic name or example for each branch.

Branch 1	Branch 2	Branch 3	Branch 4
SDKs and Tools	**Simulation and Hardware**	**Noise and Mitigation**	**Error Correction**
Example: _____	Example: _____	Example: _____	Example: _____

(Answer key — example values: Qiskit, Cirq, OpenQASM; Aer simulator, cloud QPU, transpilation; T1/T2, zero-noise extrapolation, dynamical decoupling; bit-flip code, Shor code, surface code)

🔬 Section L — Case Study / Real-World Activity

Scenario

A student team writes a Bell-state circuit in Qiskit. On a noiseless Aer simulator with 1024 shots they see only “00” and “11”, about half each. On a cloud QPU the histogram shows 00 ≈ 46%, 11 ≈ 44%, 01 ≈ 5% and 10 ≈ 5%. The backend reports T1 ≈ 100 µs, T2 ≈ 80 µs, a two-qubit gate error of about 1% and a readout error of about 2%; the two qubits used are not directly connected on the coupling map.

Questions

1. Why do “01” and “10” appear on the hardware? Identify the likely noise sources from the backend properties.

2. What does transpilation do for this circuit on this backend, and why can it increase the circuit depth?

3. Name two mitigation techniques that could improve the result and explain how a calibration matrix would help.

4. The team wants to protect a stored qubit against bit flips using the 3-qubit repetition code with a flip probability p = 0.05 per qubit. What is the logical error probability, and does the code help?

Expected Learning Outcome

Students interpret the differences between simulator and hardware results, link them to noise sources and backend properties, and choose suitable mitigation or error-correction methods.

📊 Section M — Self-Assessment

☐ **Yes** ☐ **Partly** ☐ **No** I can explain the major concepts of this unit.

☐ **Yes** ☐ **Partly** ☐ **No** I can define the important terminology from this unit.

☐ **Yes** ☐ **Partly** ☐ **No** I can solve problems/exercises from this unit.

☐ **Yes** ☐ **Partly** ☐ **No** I can apply the concepts of this unit to a new situation.

💭 Section N — Student Reflection

What did I learn in this unit?

What was difficult for me?

What do I need to practice more?

One important concept I learned:

🔑 Section O — Instructor Answer Key

(For instructor use — covers all objective and subjective items in this worksheet.)

MCQ Answers

- 1. C — Write, transpile, simulate, run on hardware, mitigate and interpret is the usual NISQ workflow.
- 2. A — OpenQASM is a text format that acts as a common intermediate representation between tools and devices.
- 3. D — A state vector has 2^n complex amplitudes, so memory doubles with each added qubit.
- 4. B — The state-vector simulator returns the amplitudes; the sampling simulator returns only counts.
- 5. A — T1 is energy relaxation from $|1\rangle$ to $|0\rangle$; dephasing is T2.
- 6. C — The bit-flip channel applies X with probability p; the phase-flip channel applies Z.
- 7. B — For I/2, $\mathrm{Tr}(\rho^2) = 1/4 + 1/4 = 1/2$, the minimum purity of a single qubit.
- 8. D — Zero-noise extrapolation measures at amplified noise (e.g. 1×, 2×, 3×) and extrapolates to 0×.
- 9. A — Encoding is $\alpha|000\rangle + \beta|111\rangle$, made with two CNOTs; option D is the phase-flip code.
- 10. B — Two or three flips cause failure: $3p^2(1 - p) + p^3 = 3p^2 - 2p^3$.

Fill in the Blanks — Answers

- 1. QASM
- 2. transpilation
- 3. T2 (dephasing)
- 4. calibration
- 5. syndrome

True/False — Answers

- 1. True
- 2. False
- 3. False
- 4. True
- 5. False

Match the Following — Answer Key

1 → c 2 → d 3 → a 4 → f 5 → g 6 → b 7 → e

Short Answer — Key Points

- 1. An n-qubit state has 2^n complex amplitudes, and a simulator stores all of them. At 16 bytes each, 30 qubits need $2^{30} \times 16$ bytes = 16 GiB, and each extra qubit doubles it.
- 2. Transpilation rewrites the circuit with the device's native gates and maps logical qubits onto physical ones, inserting SWAPs where qubits are not connected, and optimizes depth and gate count. Without it the device cannot run the circuit.
- 3. T1: time for $|1\rangle$ to relax to $|0\rangle$ by energy loss. T2: time for the phase coherence of a superposition to be lost ($T2 \le 2T1$). Other errors: gate errors, readout (measurement) errors and crosstalk.
- 4. Prepare each basis state and record what is measured to fill a calibration matrix A, where A(i, j) = P(measure i | prepared j). The measured probability vector equals A times the true one, so the corrected vector is A^{-1} times the measured one (or a constrained least-squares fit).
- 5. Encode $\alpha|0\rangle + \beta|1\rangle$ as $\alpha|000\rangle + \beta|111\rangle$ with two CNOTs. Parity checks on qubits 1–2 and 2–3 give a syndrome that identifies a single flipped qubit (00 = no error, 10 = qubit 1, 11 = qubit 2, 01 = qubit 3) without measuring the data. Apply X to the identified qubit to recover.

Descriptive Questions — Key Points

- 1. Qiskit (IBM): circuits, Aer simulators and IBM Quantum access. Cirq (Google), with TensorFlow Quantum for hybrid ML. Q# and the Azure Quantum Development Kit (Microsoft). PennyLane for hybrid workflows with automatic differentiation; Amazon Braket gives cloud access to several hardware types. OpenQASM links them. Criteria: hardware access, language, ecosystem, simulators, documentation, community. Qiskit steps: create registers and a QuantumCircuit, add gates, barriers and measurements, draw, export to OpenQASM, run on Aer and read counts or the state vector.
- 2. A simulator stores 2^n amplitudes: about 30 qubits on a laptop and more on an HPC cluster, but the doubling soon becomes impossible. Cloud QPUs are reached through jobs in a queue on a chosen backend. Backend properties: qubit count, coupling map, native gates, gate and readout errors, T1 and T2. Transpilation maps the circuit to these; then run on both and compare histograms, expecting hardware results to be noisier.
- 3. Qubits interact with the environment: T1 (relaxation) and T2 (dephasing), plus gate errors, readout errors and crosstalk. Channels: bit-flip, phase-flip, depolarizing and amplitude damping. Density matrices represent mixed states; purity $\mathrm{Tr}(\rho^2) < 1$. A noise model in a simulator applies these errors to compare noiseless and noisy runs. Mitigation: calibration matrices fix readout; ZNE extrapolates to zero noise; PEC inverts noise using quasi-probabilities (with a sampling overhead); dynamical decoupling protects idle qubits; compilation reduces depth and chooses good qubit mappings; ML-based and AI-driven methods can learn noise or find shorter circuits.
- 4. No-cloning and continuous errors prevent simple copying and majority votes, so codes use entanglement and syndrome measurements. Any single-qubit error is a combination of I, X, Z and Y = iXZ, so correcting X and Z suffices. Bit-flip code: $\alpha|000\rangle + \beta|111\rangle$; phase-flip code: Hadamards give $\alpha|{+}{+}{+}\rangle + \beta|{-}{-}{-}\rangle$; Shor's 9-qubit code concatenates them and corrects any single-qubit error. Stabilizers (e.g. Z_1Z_2 and Z_2Z_3) are operators measured to find syndromes. Surface code: a 2D lattice of data and ancilla qubits with nearest-neighbour checks, threshold near 1%; below it, larger distance suppresses logical errors exponentially, but needs hundreds to thousands of physical qubits per logical qubit. Logical rate for the 3-qubit code: $3p^2 - 2p^3$, an improvement when $p < 1/2$.

Practical Activities — Expected Outputs

- 1. Sample logic: return round(2**n * 16 / 2**30, 3); the largest n is found with a loop while 2**(n + 1) * 16 <= 64 * 2**30. The outputs are 0.016, 16.0, 512.0 and 32.
- 2. Sample logic: det = $0.95\cdot0.90 - 0.10\cdot0.05 = 0.85$; $A^{-1} = (1/0.85)\cdot[[0.90, -0.10], [-0.05, 0.95]]$; the corrected p0 = $(0.90\cdot0.61 - 0.10\cdot0.39)/0.85 = 0.6$ and p1 = 0.4.
- 3. Sample logic: def logical_error(p): return 3*p**2*(1 − p) + p**3. The outputs are (0.01, 0.000298, True), (0.1, 0.028, True) and (0.5, 0.5, False).

Concept Map — Sample Answers

- SDKs and Tools: Qiskit, Cirq, Q#, PennyLane, OpenQASM. Simulation and Hardware: Aer simulators, cloud QPUs, backend properties, transpilation. Noise and Mitigation: T1 and T2, readout error, depolarizing noise, calibration matrix, zero-noise extrapolation, dynamical decoupling. Error Correction: bit-flip code, phase-flip code, Shor code, stabilizers, surface code.

Case Study — Key Points

- 1. Readout errors (about 2%), two-qubit gate errors (about 1%, and more because extra SWAP gates are needed) and decoherence (T1, T2) flip qubits, producing the wrong outcomes 01 and 10; the ideal simulator has none of these.
- 2. It rewrites H and CNOT into the native gates, maps the logical qubits to physical ones, and inserts SWAPs because the qubits are not adjacent. The SWAPs and decompositions add gates and depth, but make the circuit executable.
- 3. Measurement error mitigation (calibration matrix) and zero-noise extrapolation; dynamical decoupling and a better qubit mapping are also valid. The calibration matrix records the readout probabilities for each basis state and is inverted to correct the measured counts.
- 4. $3p^2(1 - p) + p^3 = 3(0.0025)(0.95) + 0.000125 = 0.00725$, about 0.7%. This is below 5%, so the code helps ($p < 0.5$), at the cost of three physical qubits per logical qubit.